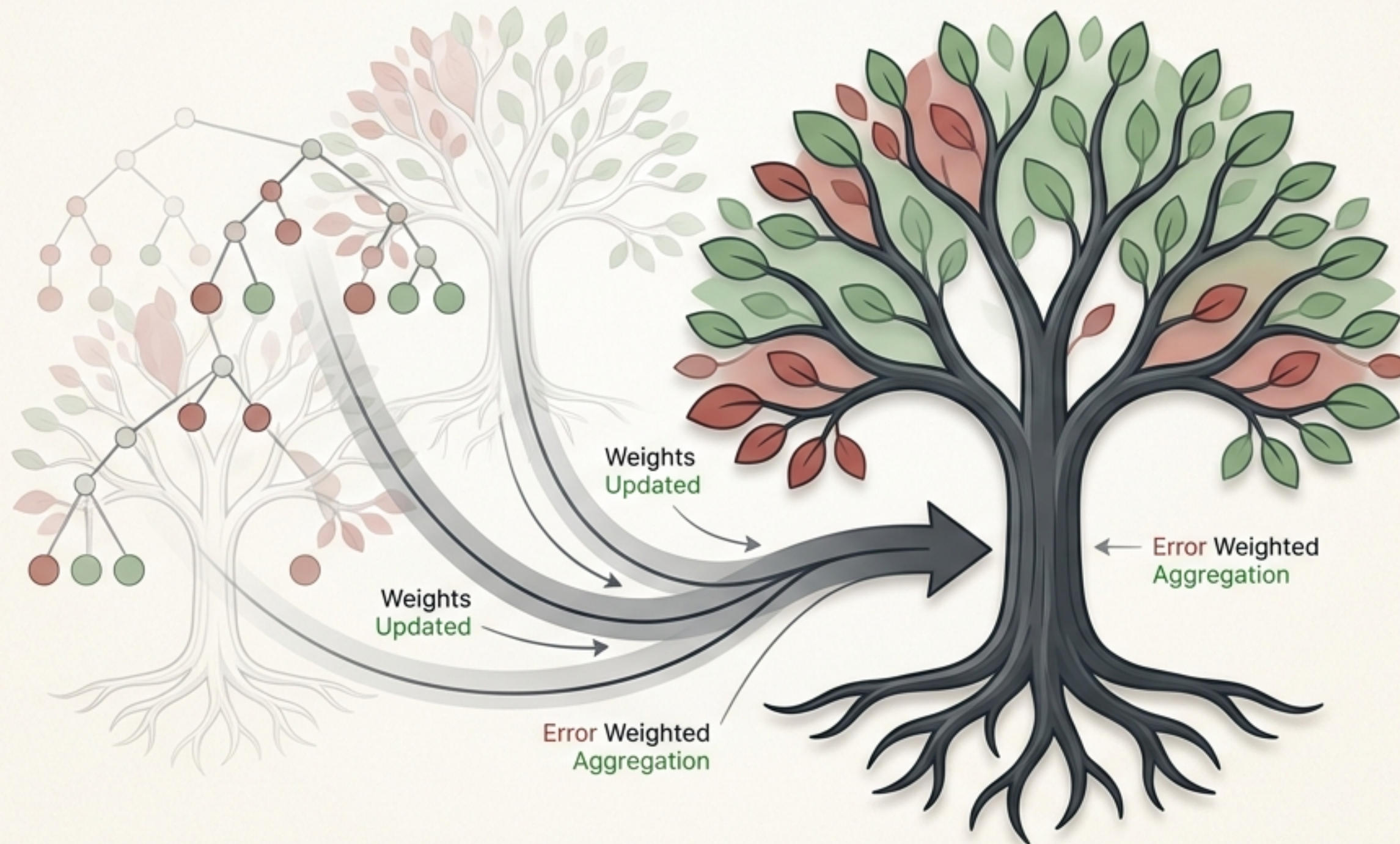


Demystifying AdaBoost

A step-by-step **visual guide** to the math and mechanics behind adaptive boosting.



The raw ingredients for our single training cycle

To understand how AdaBoost learns, we will track a tiny, hypothetical dataset of 5 rows through one complete algorithmic loop.

Row	X1	X2	y
1	2.4	1.1	1
2	3.8	0.5	0
3	1.5	2.9	1
4	4.2	1.7	1
5	0.9	3.4	0

Giving every data point an equal starting voice

Before the first model is trained, every row receives a baseline weight. The formula is simply $1/N$, ensuring all initial weights sum perfectly to 1.

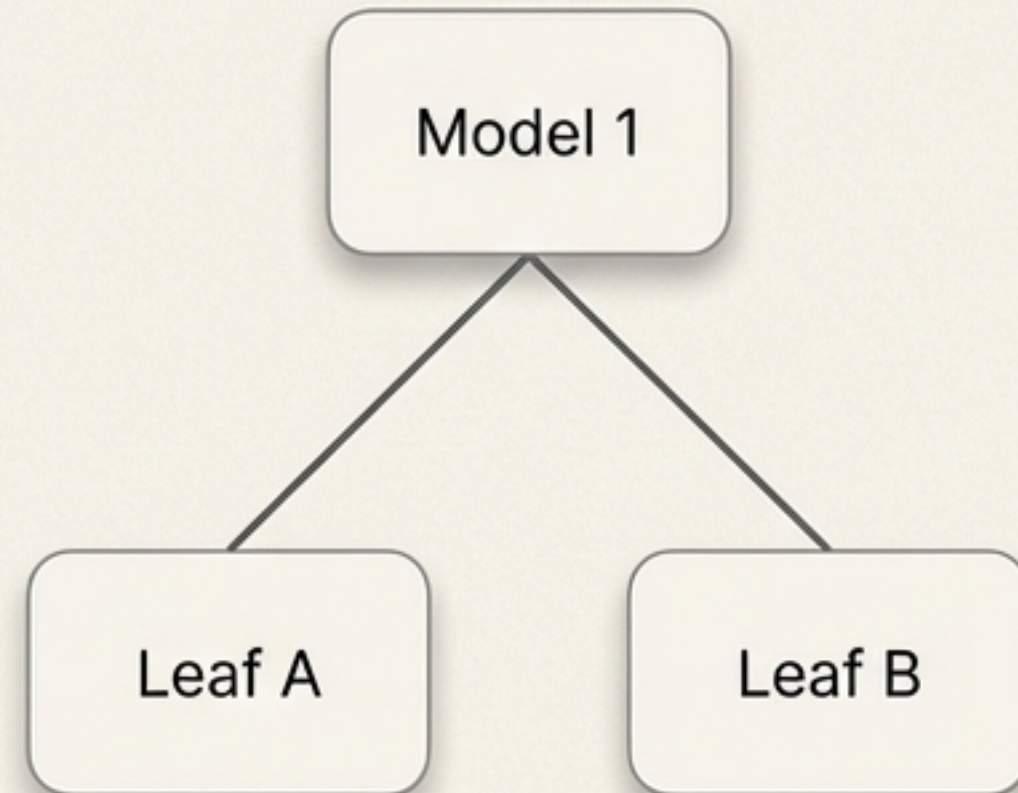
Row	X1	X2	y	Initial Weight (w)
1	2.4	1.1	1	0.2
2	3.8	0.5	0	0.2
3	1.5	2.9	1	0.2
4	4.2	1.7	1	0.2
5	0.9	3.4	0	0.2

$\frac{1}{5} = 0.2$

Sum = 1.0

The weak learner makes its first cut

We train a Decision Stump on the dataset. It chooses the split that yields the maximum decrease in entropy. It will make mistakes.

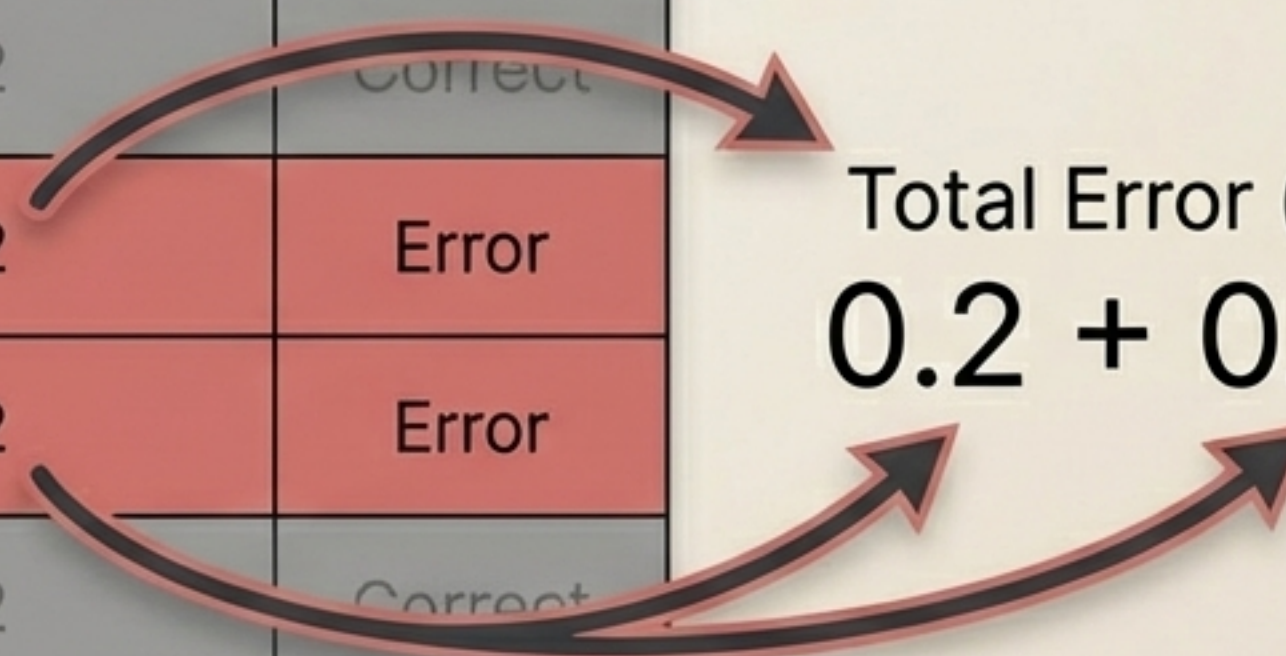


Row	X1	X2	y	Initial Weight (w)	Prediction
1	2.4	1.1	1	0.2	Correct
2	3.8	0.5	0	0.2	Error
3	1.5	2.9	1	0.2	Error
4	4.2	1.7	1	0.2	Correct
5	0.9	3.4	0	0.2	Correct

Measuring the scale of the model's failure

The total error is not just a count of mistakes. It is the sum of the weights of the misclassified rows.

Row	X1	X2	y	Initial Weight (w)	Prediction
1	2.4	1.1	1	0.2	Correct
2	3.8	0.5	0	0.2	Error
3	1.5	2.9	1	0.2	Error
4	4.2	1.7	1	0.2	Correct
5	0.9	3.4	0	0.2	Correct



Total Error (ϵ)
 $0.2 + 0.2 = 0.4$

Calculating the model's weight of authority

Based on the error, we calculate Alpha. This dictates how much say this specific model will have in the final ensemble.

The diagram illustrates the calculation of Alpha, the model's weight of authority, based on the error rate and success rate. The equation is:

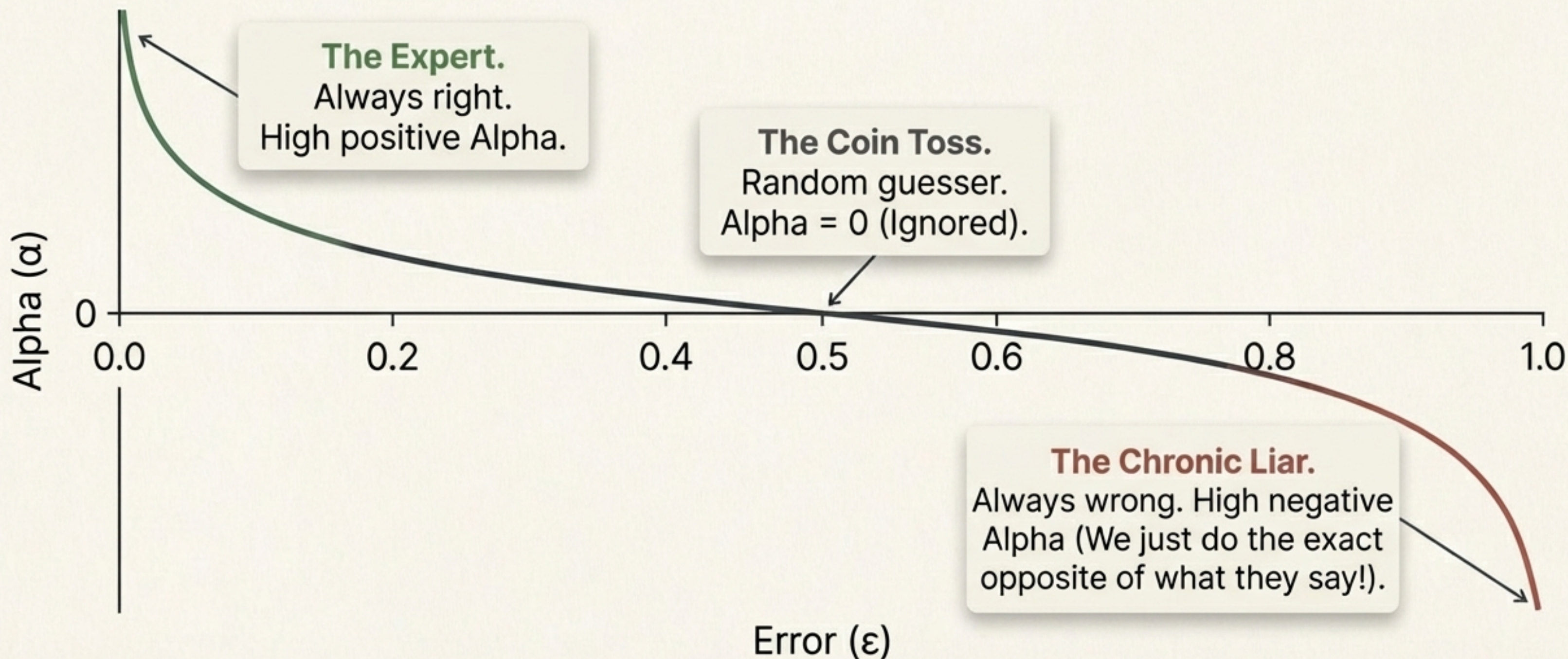
$$\alpha = \frac{1}{2} \ln\left(\frac{1 - \epsilon}{\epsilon}\right)$$

Annotations:

- Model's Voting Power** points to α .
- Natural Logarithm scales the penalty** points to $\ln$.
- Success Rate** points to $1 - \epsilon$.
- Error Rate** points to ϵ .

The logic of the chronic liar

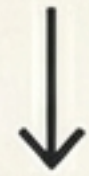
Why does the Alpha formula look like this? It rewards extreme reliability, ignores random guessing, and cleverly weaponises consistent incorrectness.



Locking in the authority for Model 1

We plug our Total Error into the formula to find out exactly how much voting power our first Decision Stump receives.

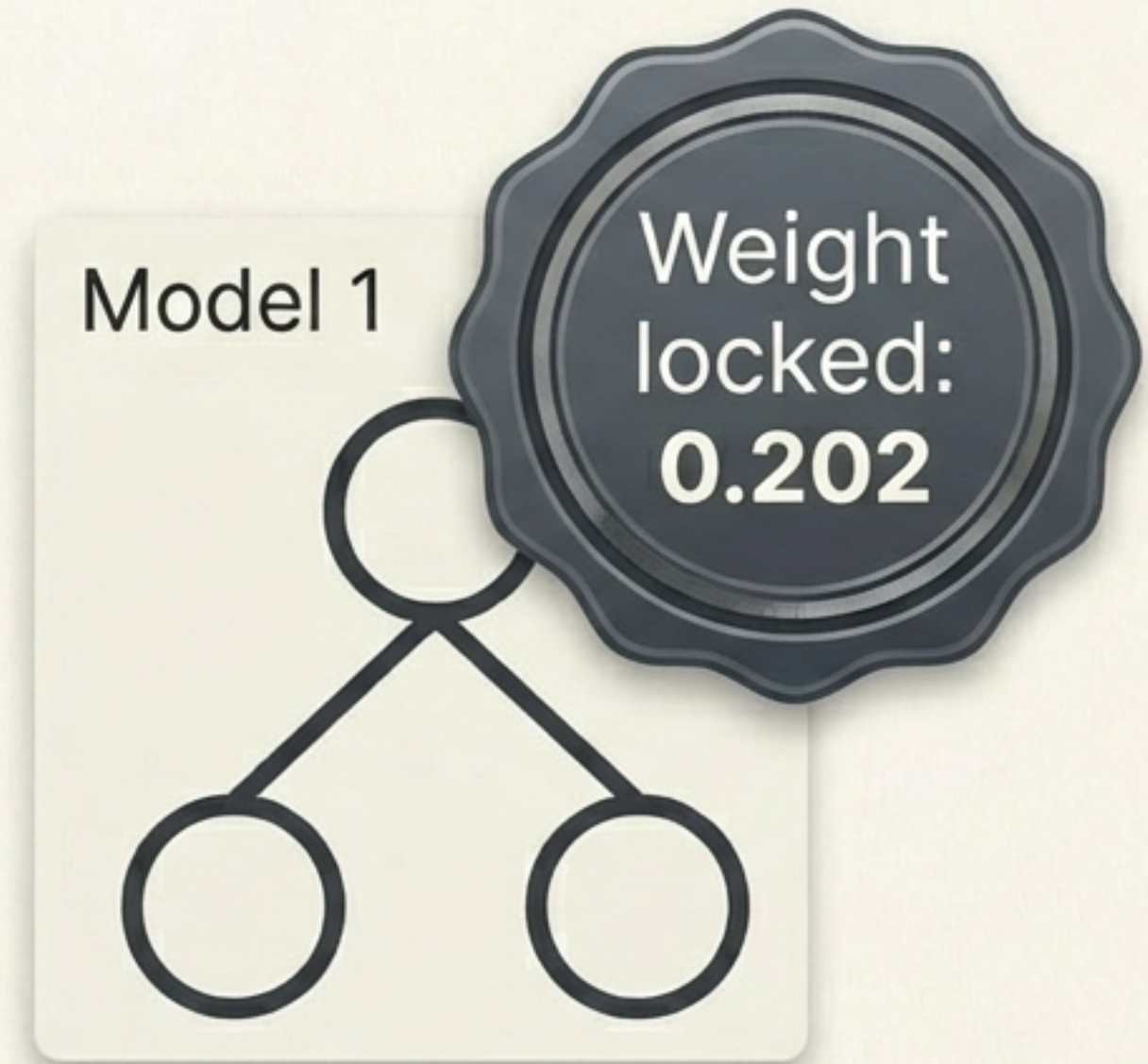
$$\alpha = \frac{1}{2} \ln\left(\frac{1 - 0.4}{0.4}\right)$$



$$\alpha = \frac{1}{2} \ln\left(\frac{0.6}{0.4}\right)$$



$$\alpha = 0.202$$



Punishing mistakes and rewarding success

We must force the next model to focus on the current model's mistakes. We do this by boosting the weights of misclassified rows and shrinking the weights of correct ones.

Misclassified (Rows 2 & 3)

$$W_{\text{new}} = W_{\text{old}} * e^{\alpha}$$

$$0.2 * e^{(0.202)} = 0.24$$



Correctly Classified (Rows 1, 4 & 5)

$$W_{\text{new}} = W_{\text{old}} * e^{-\alpha}$$

$$0.2 * e^{(-0.202)} = 0.16$$



Normalising the new weights back to a perfect whole

Because we increased some weights and decreased others, they no longer sum to 1. We must divide each weight by the new total sum to normalise them.

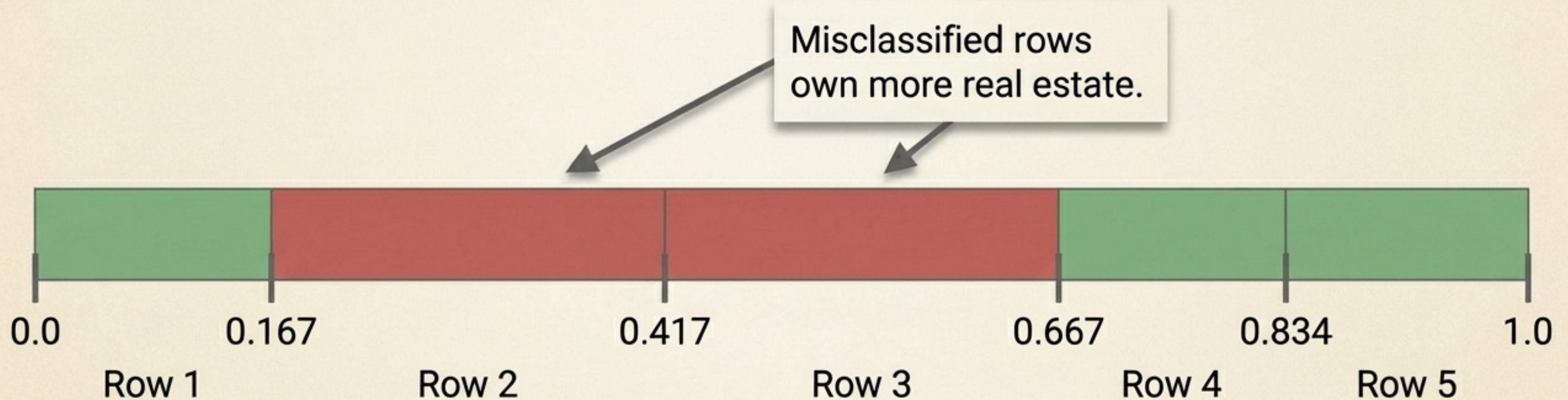
Row	Raw New Weight		Final Normalised Weight
Row 1	0.16	divide by 0.96 →	0.167
Row 2	0.24		0.250
Row 3	0.24		0.250
Row 4	0.16		0.167
Row 5	0.16		0.167

$$\left[\begin{array}{l} (3 * 0.16) + (2 * 0.24) \\ = 0.96 \end{array} \right]$$

$$\left[\text{Sum} = 1.0 \right]$$

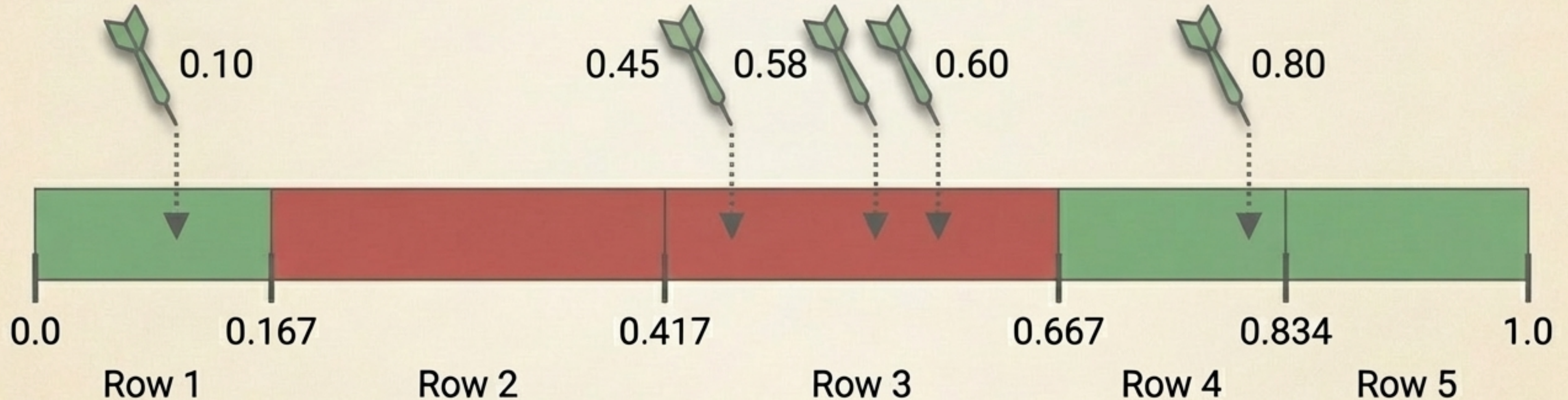
Mapping weights to probability buckets

To create our next dataset, we map our normalised weights onto a cumulative range from 0.0 to 1.0. Larger weights command wider ranges.



Throwing random darts to pick the next dataset

We generate 5 random numbers between 0 and 1. Because the misclassified rows own wider buckets, they are statistically caught more often.



A new dataset, obsessed with past mistakes

The up-sampling generates a brand new dataset. It discards rows that were easy to predict and heavily duplicates the rows the previous model struggled with.

Original Data

Row 1
Row 2
Row 3
Row 4
Row 5



New Data for Model 2

Row 1
Row 3
Row 3
Row 3
Row 4

Model 2 is forced to learn from Model 1's errors.

Passing the baton

This new, mistake-heavy dataset is passed to Model 2. The entire process repeats T times. The final prediction combines every model's output, scaled by their Alpha.

